

Hirschmann / Belden — Agentic Pentesting: Interview Practice Sheet

Interview: **Tue 15 Sep 2026, 13:30**, remote (~90 min), live case study expected. Panel: Sagerer (Security Testing) · Madbouly (Validation Automation) · Bechtel (Innovation Architect) · Rachinger (People).

How to use this: these are talking points, not scripts. Read each answer, then say it in your own words out loud. Keep answers short in the room — 30 to 60 seconds each — and stop so they can dig. Where a weakness is listed, say it yourself before they find it; being honest about a flaw is a strength for an evaluation role.

0. Self-introduction

English

Hi, I'm Rahul Rawat. Over the last one and a half years, I've gone deep into AI and data science. For example, I worked with eRay on a case study where we detected algal blooms about two weeks in advance, so they can be anticipated early. As a personal project, I built a multi-agent RAG system that answers questions from policy PDFs and also guards the output — it redacts sensitive information and blocks responses to prompt-injection queries. Recently, I was part of a research group at KIT, where we collected data from different participants. The goal of the study was to look at how vehicles approach an intersection at a red light at different speeds — we measured eye movements, heart rate, and acceleration. I took part as a volunteer student to support the study. That's it from my side.

German (B1)

Hallo, ich bin Rahul Rawat. In den letzten anderthalb Jahren habe ich mich intensiv mit KI und Data Science beschäftigt. Zum Beispiel habe ich mit eRay an einer Fallstudie gearbeitet. Dort haben wir Algenblüten etwa zwei Wochen im Voraus erkannt, damit man sie früh vorhersagen kann. Als persönliches Projekt habe ich ein Multi-Agenten-RAG-System gebaut. Es beantwortet Fragen aus Richtlinien-PDFs und schützt auch die Ausgabe: Es entfernt sensible Informationen und blockiert Antworten auf Prompt-Injection-Anfragen. Vor Kurzem war ich Teil einer Forschungsgruppe am KIT. Dort haben wir Daten von verschiedenen Teilnehmern gesammelt. Das Ziel der Studie war, zu untersuchen, wie Fahrzeuge bei Rot mit unterschiedlichen Geschwindigkeiten auf eine Kreuzung zufahren. Wir haben die Augenbewegungen, den Herzschlag und die Beschleunigung gemessen. Ich habe als freiwilliger Student mitgeholfen. Das war es von meiner Seite.

1. Motivation & fit

Q1 — Why agentic pentesting? Why an agent and not just better scanners? A scanner runs fixed checks and dumps a long list — no judgement, no prioritisation. A human pentester reasons about what actually matters but doesn't scale. An agent sits in between: it orchestrates the scanners, reasons about which findings matter for this specific target, and drafts the report — with a human approving anything risky. That mix of orchestration plus evaluation is exactly what I've already built in my RAG work, so it's a natural next step for me.

Q2 — Why Hirschmann, why OT and not IT? Two reasons. First, OT security is higher-stakes and less solved than IT — you can't just patch and reboot a running plant, so safe, careful automation matters more. Second, my instinct is fail-closed: in my eRay pipeline I built gate checks that halt on bad data instead of letting it flow downstream. That temperament fits a security team working near live hardware.

Q2b — Why a thesis and not a normal job? A thesis gives me six

months to build and evaluate one hard system properly, with your team's guidance. That's research depth I can't get by shipping features in a normal Werkstudent role.

2. Security fundamentals (your thinnest area — study these hardest)

Q3 — Walk me through the pentest lifecycle. Where does an agent help? Five stages: recon (gather info) → scanning / enumeration (find live hosts, open ports, services, versions — Nmap) → exploitation (use a weakness to get in) → post-exploitation (impact, pivot, persistence) → reporting. An agent is safest and most useful in stages 1 and 2 and in drafting stage 5. Exploitation I'd keep behind a human gate.

Q4 — Difference between recon, scanning, and exploitation? Recon is information gathering — passive or active — about who and what the target is. Scanning is actively probing for open ports, services and versions. Exploitation actually uses a found weakness to gain access. The first two are just information; the third changes the system's state — that's the line where a human must approve.

Q5 — Why is OT pentesting different from a web app? What can go wrong on a live switch? The targets are switches, PLCs and industrial protocols like Modbus and PROFINET — often unauthenticated by design. Availability is the top priority, not confidentiality: you cannot crash a running line. Many devices are fragile, so even an aggressive scan can knock them over. So the agent has to be read-only where possible, careful with timing, and it must never run an exploit unsupervised. The Purdue model is the standard reference for separating IT from OT layers.

Q6 — What does Nmap do? What does a vulnerability scanner add? Nmap is a network mapper — host discovery plus port, service and version scanning. `-sV` gives service versions, `-O` fingerprints the OS. A vulnerability scanner like Nessus or OpenVAS takes those service-plus-version results and matches them against known CVEs

automatically, then scores severity with CVSS. Nmap finds what's there; the scanner says what's known to be wrong with it.

Vocabulary to have ready: CVE = a catalogued vulnerability; CVSS = its 0–10 severity score; CIA = Confidentiality, Integrity, Availability; whitebox = testing with internal/source access; blackbox = no inside knowledge; least privilege = grant only the access needed.

3. Your agentic work (defend the architecture — own the flaws)

Q7 — Your GuardrailsAgent runs at the END. When is an injected query caught, and is that right? Honest answer: in my current version it runs at the end, so an injected query has already gone through retrieval and generation before it's caught — the guard only stops the tainted answer from being shown. That's a design flaw. Injection detection belongs at the input, before generation. I built the enforcement wiring — block plus skip-logging, which is the hard part — and moving the check to the front is the first correction I'd make.

Q8 — Your injection defense is a keyword blocklist. How would I bypass it? Easily — paraphrase the instruction, translate it into another language, or obfuscate it, and a six-phrase keyword list sails right through. It also false-positives on innocent substrings. A real defense uses an LLM- or classifier-based detector that judges intent rather than exact words, plus input/output separation and least-privilege on what the model is actually allowed to do. Knowing keyword matching is insufficient is the point — I would never ship it as robust.

Q9 — Your Verifier produces metrics but never blocks. Is that really verifying? Fair criticism. Right now the pipeline is linear — the Verifier computes alignment, factuality and citation checks, but it logs, it doesn't block or re-route. So today it's observability, not enforcement. To make it a real verifier I'd add a conditional edge: below a factuality threshold, re-run generation or fall back to "not found" instead of showing the answer. I'd describe the current version honestly as measurement, with the gate

as the next step.

Q10 — Why is your judge temperature-locked but your generator isn't? Why does it matter? The judge runs at temperature 0, so it's deterministic — the same answer scores the same every time, which is what makes evaluation runs comparable across changes. My generator and debate agents aren't locked, so their wording varies run to run. For a pentest agent this matters a lot: if the same target produces different findings each run, you can't trust or reproduce results. So I'd lock everything on the evaluation and decision path and allow non-determinism only where it doesn't affect the verdict.

4. The core case study

Q11 — "Design an AI agent that pentests our managed switches — web interface and firmware. Walk us through the architecture, how it decides what to test, how you keep it safe on live hardware, and how you'd prove it beats a human or a plain scanner."

Say this first: *"Before I design, let me clarify a few things — are the targets IT or OT, is this a lab range or live hardware, which pentest stage do you want automated, and is the goal to augment your analysts or run fully autonomous?"* Then walk the 7 beats:

1. **Scope** — targets (their OT switches: web mgmt interface + firmware), which stages (recon + scanning + triage, not autonomous exploitation), and augment-analyst mode, not full auto.
2. **Current state** — the manual workflow today and where the bottleneck is (usually triaging a flood of scanner findings).
3. **Architecture** — an orchestrator/router (like my Hybrid RAG router that picks SEARCH / WEB / DIRECT) → tool adapters behind a clean seam (Nmap, a vuln scanner) → an LLM/RAG step that prioritises and plausibilises findings (this is my JudgeAgent reframed: is this finding real, exploitable, correctly matched to a CVE?).
4. **Guardrails** — scoping limits, sandboxing, no state-changing

- action without approval, PII/sensitive-data redaction in reports.
5. **Human gate** — nothing is "confirmed" and nothing intrusive runs until an analyst signs off; the agent drafts, the human approves. (This is my CreditIQ human-in-the-loop, EU AI Act Art. 14.)
 6. **Evaluation** — a known-vulnerable test range for ground truth; a manual pentest as the baseline; then precision/recall on findings, reproducibility across runs, and time saved.
 7. **Risks** — hallucinated findings, non-determinism, dangerous false negatives, data sensitivity, and crashing fragile OT devices.

Q12 — How do you stop the agent hallucinating a vulnerability that isn't there? Three layers. First, ground every finding in real tool output — the agent cites the Nmap/scanner evidence for each claim, the same way my RAG stamps citations from metadata so it can't invent a source. Second, a verification step that cross-checks the CVE against the actual detected service version before reporting. Third, a human approval gate — nothing is confirmed until an analyst signs off. And I'd report the false-positive rate explicitly, the way I report FPR in CreditIQ.

Q13 — How would you evaluate it? Baseline and metrics? Build a test range of known-vulnerable targets so I have ground truth. Run a manual pentest as the baseline. Then measure the agent against it: precision and recall on findings — false positives waste analyst time, false negatives are dangerous — plus reproducibility (same target, same findings across runs) and time saved. That's the same shape as my EvalAgent computing metrics on a labelled set, and my thesis experience of choosing ROC-AUC so a baseline can't hide minority-class errors applies directly.

Q14 — What if the agent wants to run an exploit that could crash a production switch? Then it does not run it. My hard rule: no intrusive or state-changing action without explicit human approval, and on OT, read-only by default. The agent can recommend the exploit, explain the risk and expected impact, and queue it for an analyst — but it never pulls the trigger itself. In OT, availability wins over thoroughness. This is exactly the human-in-the-loop design I already shipped in CreditIQ for Article 14 oversight.

5. German check (expect 1–2 short ones, then back to English)

Warum interessiert dich Cybersicherheit?

Cybersicherheit interessiert mich, weil sie wichtig ist und nie langweilig wird. In meinem RAG-Projekt habe ich schon Guardrails gegen Prompt-Injection gebaut. Jetzt möchte ich das auf echte Systeme wie Ihre Switches anwenden.

Erzähl mir kurz von deinem wichtigsten Projekt.

Mein wichtigstes Projekt ist ein Multi-Agenten-RAG-System. Neun Agenten arbeiten nacheinander: Sie planen die Frage, suchen Dokumente, schreiben eine Antwort und prüfen sie. Am Ende gibt es Guardrails, die sensible Daten entfernen. Ich habe es lokal mit Ollama gebaut, ohne API-Key.

Tip: if a German technical word is hard (Prompt-Injection, Guardrails), just say it in English — mixing is normal and expected.

6. Extra case studies to rehearse

Same method every time: **clarify first** → **frame the structure out loud** → **think aloud** → **anchor in your projects** → **surface safety** → **manage time**. They grade how you think, not the perfect answer.

Case Study B — Finding triage / prioritisation

"Our nightly scan of the switch fleet produces hundreds of findings. Design an agent that triages and prioritises them and drafts a report."

- Clarify: how many devices, what scanner, who reads the report, what "priority" means to them (CVSS only, or exposure + business impact?).
- Approach: ingest scanner output → dedupe and group by device/

service → LLM ranks by CVSS **plus context** (is the port actually exposed? is the service reachable from IT?) → plausibilise each (cross-check CVE vs detected version, drop likely false positives) → draft a human-readable report with evidence citations → analyst reviews.

- This is almost literally your JudgeAgent + citation story. Lead with it. Metric: does the agent's top-10 match the analyst's top-10 (precision@10)?

Case Study C — Prove it works (Madbouly's angle, pure evaluation)

"How would you prove our agent is actually better than our current manual pentest?"

- Clarify: what does "better" mean to them — more findings, fewer misses, faster, cheaper?
- Approach: define ground truth on a known-vuln test range → run manual baseline and agent on the same targets → compare precision/recall on findings, time-to-report, and reproducibility (run the agent 5× on the same target — do findings stay stable?) → report an honest confusion-matrix view, not a single flattering number.
- Anchor: your thesis lesson (ROC-AUC over accuracy so a baseline can't hide minority errors) and CreditIQ (reporting an uncomfortable FPR honestly). Weakness to name: non-determinism hurts reproducibility, which is why you'd temperature-lock the decision path.

Case Study D — Firmware regression testing (OT safety focus)

"A new switch firmware ships every two weeks. Design an agent that checks each release for known vulnerability classes — safely."

- Clarify: lab devices or production, is there a test bench, which vuln classes matter most (default creds, insecure protocols, exposed mgmt interface, unpatched CVEs).
- Approach: run on an isolated test bench, never production → agent runs the same read-only checklist each release (default credentials, Telnet/plain Modbus enabled, TLS present, open mgmt ports) → diff findings against the previous release to flag regressions → everything read-only, exploit attempts queued for

human approval → reproducible run every release.

- Safety line to say: "In OT the whole value is a repeatable, read-only regression check on a bench — I'd rather miss an exotic finding than risk a device."
-

7. Fast rules for the room

- Lead with the **evaluation** story — it's your rarest strength.
- Keep the two RAG projects straight: **Hybrid RAG** = router + memory + live web; **Multi-Agent RAG (PolicyNeural)** = multilingual, LLM-as-Judge, guardrails, no memory.
- Surface **safety first** on any case study: human-in-the-loop, no unsupervised exploits, read-only on OT.
- On security depth you don't have: "**I don't know — here's how I'd find out.**" Never bluff.
- State your German once, confidently: **B1, in progress**, happy in English or a mix. Don't apologise for it.
- If asked about the 0.98 bachelor-thesis AUC: **distrust it out loud** — likely duplicate rows + aggressive outlier removal; the honest figure is ~80%. Catching your own inflated number is the best answer for an eval role.